

Reviewer Pack — Immanant Positivity Width Bounds Monotone KW Depth for Perfe...

Entry 15b45d9e4966 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-17 04:06:38 UTC

Immanant Positivity Width Bounds Monotone KW Depth for Perfect Matching

Entry ID: 15b45d9e4966 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-17 04:06:38 UTC

1. Conjecture

Field A (mathematical branch): Stembridge–Stanley immanant theory: the λ -immanants $\text{imm}_\lambda(S) = \sum_{\sigma \in S_n} \chi_\lambda(\sigma) \prod_i S[i, \sigma(i)]$ organise matchings of a 0/1 matrix into Schur–Weyl S_n -isotypic cells (a corner of symmetric-group representation theory with very few papers connecting it to monotone circuit lower bounds outside of GCT speculation) **Field B** (complexity object): Monotone Karchmer–Wigderson tree depth $d_{\text{KW}}(S)$ at a 1-input S of the bipartite perfect-matching function PM_n , equivalent by KW to $\log_2$ of the local monotone formula leaf size restricted to matchings inside S vs. small bipartite vertex covers

Statement:

For $3 \leq n \leq 6$ and any $n \times n$ 0/1 matrix S with $\text{perm}_n(S) \geq 1$, let $\Sigma(S) := \{\sigma \in S_n : S[i, \sigma(i)] = 1 \ \forall i\}$ and define the immanant positivity width $w(S) := \#\{\lambda \vdash n : \text{imm}_\lambda(S) > 0\}$. Let $d_{\text{KW}}(S)$ be the minimum depth of any KW protocol that on Alice-input $\tau \in \Sigma(S)$ and Bob-input C , an $(n-1)$ -vertex cover of the bipartite complement of some 0-input S' , outputs an edge $(i, j) \in \tau \cap C$. Conjecture: $d_{\text{KW}}(S) \geq \lceil \log_2 w(S) \rceil$ for every such S , with equality only when $\Sigma(S)$ is RSK-shape-monochromatic; in particular $w(I_n) = p(n)$, forcing $d_{\text{KW}}(I_n) \geq \lceil \log_2 p(n) \rceil = \Theta(\sqrt{n})$, a representation-theoretic obstruction in the spirit of GCT’s perm/det separation.

Rationale (proposer’s reasoning):

Each rectangle of a KW protocol pools matchings that all hit a common edge of Bob’s cover; we conjecture this pooling is incompatible with carrying more than one irreducible χ_λ -component, so the number of non-vanishing λ -immanants is a Schur–Weyl obstruction to small KW trees. This mirrors the GCT philosophy that perm_n ’s orbit closure is Schur-rich while det_m ’s is sign-isotypic: a small substitution-stable Schur-width invariant would translate directly into a perm-vs-det monotone gap. The link of immanants to KW depth is, to our knowledge, untouched in the literature.

Taxonomy category: KARCHMER_WIGDERSON (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c0f750e923e36480

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 360 matrices ($n \in \{3,4,5,6\}$, 3 densities, 30 seeds), conjecture SUPPORTED iff $d_KW(S) \geq \lceil \log_2 w(S) \rceil$ holds for $\geq 95\%$ of matrices AND $\text{mean}(d_KW - \lceil \log_2 w \rceil) \geq 0$ AND OLS slope of d_KW on $\log_2 w(S) \geq 0.8$ with $R^2 \geq 0.5$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.86	SAFE	SAFE
ALGEBRIZATION	SAFE	0.85	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - immanant Karchmer-Wigderson monotone perfect matching - Stembridge immanant positivity bipartite permanent monotone formula lower bound-geometric complexity theory immanant Schur-Weyl monotone circuit perfect matching

Top relevant hits considered: - [<http://arxiv.org/abs/2107.05128v2>] Karchmer-Wigderson Games for Hazard-free Computation - [<http://arxiv.org/abs/2002.07444v1>] Multiparty Karchmer-Wigderson Games and Threshold Circuits - [<http://arxiv.org/abs/2003.09602v2>] The Number of Perfect Matchings in Möbius Ladders and Prisms - [<http://arxiv.org/abs/2304.05285v1>] Hook-Shape Immanant Characters from Stanley-Stembridge Characters - [<http://arxiv.org/abs/1801.08709v2>] Adaptive Lower Bound for Testing Monotonicity on the Line - [<http://arxiv.org/abs/0906.0693v3>] An improved lower bound on the counterfeit coins problem - [<http://arxiv.org/abs/1904.05483v2>] Parallels Between Phase Transitions and Circuit Complexity? - [<http://arxiv.org/abs/1611.00827v2>] Geometric complexity theory and matrix powering

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import sys
import random
import math
import itertools
from collections import defaultdict

def generate_random_matrix(n, rho, seed):
    random.seed(seed)
    matrix = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            if random.random() < rho:
```

```

        matrix[i][j] = 1
    return matrix

def perm_n(matrix):
    n = len(matrix)
    count = 0
    for perm in itertools.permutations(range(n)):
        valid = True
        for i in range(n):
            if matrix[i][perm[i]] == 0:
                valid = False
                break
        if valid:
            count += 1
    return count

def cycle_type(perm):
    visited = [False] * len(perm)
    cycles = []
    for i in range(len(perm)):
        if not visited[i]:
            cycle = []
            j = i
            while not visited[j]:
                visited[j] = True
                cycle.append(j)
                j = perm[j]
            cycles.append(tuple(sorted(cycle)))
    return tuple(sorted(cycles))

def murnaghan_nakayama(matrix, n):
    imm = defaultdict(int)
    for perm in itertools.permutations(range(n)):
        ct = cycle_type(perm)
        imm[ct] += 1
    return imm

def w(S):
    n = len(S)
    imm = murnaghan_nakayama(S, n)
    return len(imm)

def generate_covers(n, max_covers):
    covers = []
    for _ in range(max_covers):
        cover = set()
        for i in range(n - 1):
            cover.add((i, random.randint(0, n - 1)))
        covers.append(cover)
    return covers

def kw_depth(S, covers):
    n = len(S)
    Sigma = []
    for perm in itertools.permutations(range(n)):
        valid = True

```

```

    for i in range(n):
        if S[i][perm[i]] == 0:
            valid = False
            break
    if valid:
        Sigma.append(perm)

if not Sigma:
    return 0

depth = 0
current_level = [Sigma]
while True:
    next_level = []
    for group in current_level:
        if len(group) == 1:
            continue
        for cover in covers:
            subsets = defaultdict(list)
            for sigma in group:
                hit = False
                for (i, j) in cover:
                    if sigma[i] == j:
                        hit = True
                        break
                if hit:
                    subsets[cover].append(sigma)
            if len(subsets) > 1:
                for subset in subsets.values():
                    next_level.append(subset)
                break
    if not next_level:
        break
    current_level = next_level
    depth += 1
return depth

def run_trial(seed):
    n_values = [3, 4, 5, 6]
    rho_values = [0.4, 0.6, 0.8]
    total_instances = 0
    total_support = 0
    metric_values = []
    counterexamples = []

    for n in n_values:
        for rho in rho_values:
            matrix = generate_random_matrix(n, rho, seed)
            if perm_n(matrix) == 0:
                continue

            total_instances += 1
            w_S = w(matrix)
            covers = generate_covers(n, 1500)
            d_KW = kw_depth(matrix, covers)

```

```

        metric_value = d_KW - math.ceil(math.log2(w_S))
        metric_values.append(metric_value)

        if d_KW < math.ceil(math.log2(w_S)):
            counterexamples.append((matrix, n, rho, seed))

        if d_KW >= math.ceil(math.log2(w_S)):
            total_support += 1

    if total_instances == 0:
        return {
            "metric_name": "d_KW - log2(w(S))",
            "metric_value": 0,
            "instances_tested": 0,
            "conjecture_holds": False,
            "counterexample": "No valid matrices generated"
        }

    mean_metric = sum(metric_values) / len(metric_values)
    std_metric = math.sqrt(sum((x - mean_metric) ** 2 for x in metric_values) / len(metric_values))
    support_fraction = total_support / total_instances

    if counterexamples:
        counterexample = f"Matrix: {counterexamples[0][0]}, n: {counterexamples[0][1]}, rho: {counterexamples[0][2]}, seed: {counterexamples[0][3]}"
    else:
        counterexample = ""

    return {
        "metric_name": "d_KW - log2(w(S))",
        "metric_value": mean_metric,
        "instances_tested": total_instances,
        "conjecture_holds": support_fraction >= 0.95,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]

    total_instances = 0
    total_support = 0
    metric_values = []
    counterexamples = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        total_instances += result["instances_tested"]
        if result["conjecture_holds"]:
            total_support += 1
        metric_values.append(result["metric_value"])
        if result["counterexample"]:
            counterexamples.append(result["counterexample"])

    if total_instances == 0:
        print("RESULT: INCONCLUSIVE reason=no_valid_matrices")
        sys.exit(0)

```

```

mean_metric = sum(metric_values) / len(metric_values)
std_metric = math.sqrt(sum((x - mean_metric) ** 2 for x in metric_values) / len(metric_values))
support_fraction = total_support / len(seeds)

if counterexamples:
    print(f"RESULT: FALSIFIED counterexample={counterexamples[0]} first_failing_seed={seeds[counterexamples[0]]}")
elif support_fraction >= 0.95 and mean_metric >= 0:
    print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fraction}")
else:
    print("RESULT: INCONCLUSIVE reason=insufficient_support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_70051383.py", line 178, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_70051383.py", line 132, in run_trial
    d_KW = kw_depth(matrix, covers)
            ^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_70051383.py", line 104, in kw_depth
    subsets[cover].append(sigma)
    ~~~~~^~~~~~
TypeError: unhashable type: 'set'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `TypeError` ('unhashable type: set') in `kw_depth` before producing any data, so no metrics were computed and the pre-registered support condition cannot be evaluated. | next: Fix the `kw_depth` implementation by using a hashable key (e.g., `frozenset`) for cover keys in the subsets dict, then rerun the 360-matrix sweep.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	221568
2	preregistration	claude_max	opus	0	0	7021
3	novelty	claude_max	opus	0	0	4293
4	novelty	claude_max	opus	0	0	8177
5	test_gen	mistral	codestral-latest	0	0	310867
6	test_gen	mistral	codestral-latest	0	0	310268
7	test_gen	mistral	codestral-latest	0	0	311519
8	test_gen	mistral	codestral-latest	0	0	309621
9	judge	claude_max	opus	0	0	4058

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 1487390 ms total latency. Provider mix: {'claude_max': 5, 'mistral': 4}

(full prompt+response transcripts available in *research/audit/15b45d9e4966.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/15b45d9e4966.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/15b45d9e4966.tar.gz* (if generated)